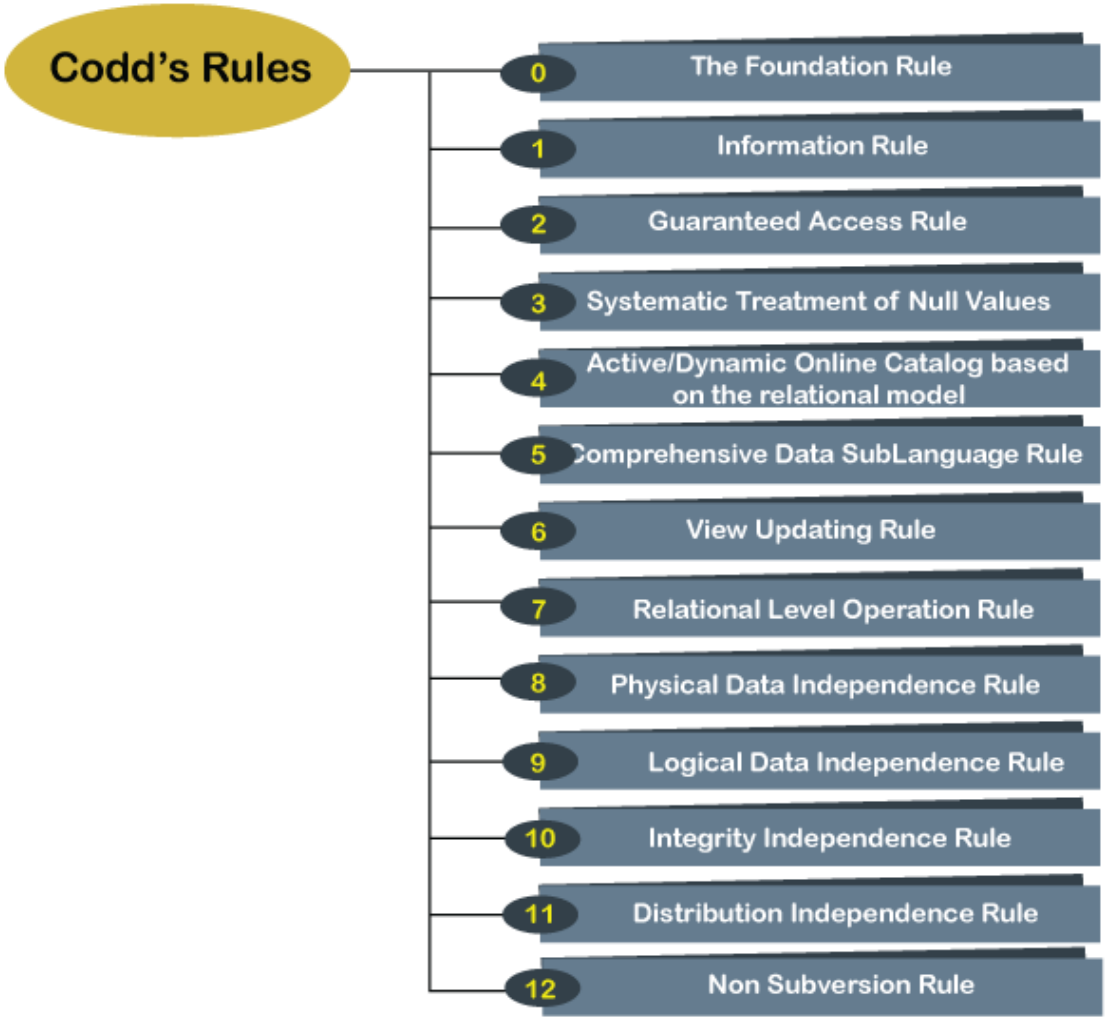
SUBJECT:PLSQL PROGRAMMING

UNIT-1

PROF SMITA M MULEY

12 Codd's Rules

Every database has tables, and constraints cannot be referred to as a rational database system. And if any database has only relational data model, it cannot be a **Relational Database System (RDBMS)**. So, some rules define a database to be the correct RDBMS. These rules were developed by **Dr. Edgar F. Codd (E.F. Codd)** in **1985**, who has vast research knowledge on the Relational Model of database Systems. Codd presents his 13 rules for a database to test the concept of DBMS against his relational model, and if a database follows the rule, it is called a **true relational database (RDBMS)**. These 13 rules are popular in RDBMS, known as **Codd's 12 rules**.



Rule 0: The Foundation Rule

The database must be in relational form. So that the system can handle the database through its relational capabilities.

Rule 1: Information Rule

A database contains various information, and this information must be stored in each cell of a table in the form of rows and columns.

Rule 2: Guaranteed Access Rule

Every single or precise data (atomic value) may be accessed logically from a relational database using the combination of primary key value, table name, and column name.

Rule 3: Systematic Treatment of Null Values

This rule defines the systematic treatment of Null values in database records. The null value has various meanings in the database, like missing the data, no value in a cell, inappropriate information, unknown data and the primary key should not be null.

Rule 4: Active/Dynamic Online Catalog based on the relational model

It represents the entire logical structure of the descriptive database that must be stored online and is known as a database dictionary. It authorizes users to access the database and implement a similar query language to access the database.

Rule 5: Comprehensive Data SubLanguage Rule

The relational database supports various languages, and if we want to access the database, the language must be the explicit, linear or well-defined syntax, character strings and supports the comprehensive: data definition, view definition, data manipulation, integrity constraints, and limit transaction management operations. If the database allows access to the data without any language, it is considered a violation of the database.

Rule 6: View Updating Rule

All views table can be theoretically updated and must be practically updated by the database systems.

Rule 7: Relational Level Operation (High-Level Insert, Update and delete) Rule

A database system should follow high-level relational operations such as insert, update, and delete in each level or a single row. It also supports union, intersection and minus operation in the database system.

Rule 8: Physical Data Independence Rule

All stored data in a database or an application must be physically independent to access the database. Each data should not depend on other data or an application. If data is updated or the physical structure of the database is changed, it will not show any effect on external applications that are accessing the data from the database.

Rule 9: Logical Data Independence Rule

It is similar to physical data independence. It means, if any changes occurred to the logical level (table structures), it should not affect the user's view (application). For example, suppose a table either split into two tables, or two table joins to create a single table, these changes should not be impacted on the user view application.

Rule 10: Integrity Independence Rule

A database must maintain integrity independence when inserting data into table's cells using the SQL query language. All entered values should not be changed or rely on any external factor or application to maintain integrity. It is also helpful in making the database-independent for each front-end application.

Rule 11: Distribution Independence Rule

The distribution independence rule represents a database that must work properly, even if it is stored in different locations and used by different end-users. Suppose a user accesses the database through an application; in that case, they should not be aware that another user uses particular data, and the data they always get is only located on one site. The end users can access the database, and these access data should be independent for every user to perform the SQL queries.

Rule 12: Non Subversion Rule

The non-submersion rule defines RDBMS as a SQL language to store and manipulate the data in the database. If a system has a low-level or separate language other than SQL to access the database system, it should not subvert or bypass integrity to transform data.

Introduction to SQL

- SQL stands for Structured Query Language. It is used for storing and managing data in relational database management system (RDMS).
- It is a standard language for Relational Database System. It enables a user to create, read, update and delete relational databases and tables.
- All the RDBMS like MySQL, Informix, Oracle, MS Access and SQL Server use SQL as their standard database language.
- SQL allows users to query the database in a number of ways, using English-like statements.

Rules:

SQL follows the following rules:

- Structure query language is not case sensitive. Generally, keywords of SQL are written in uppercase.
- Statements of SQL are dependent on text lines. We can use a single SQL statement on one or multiple text line.
- Using the SQL statements, you can perform most of the actions in a database.
- SQL depends on tuple relational calculus and relational algebra.

Characteristics of SQL

- SQL is easy to learn.
- SQL is used to access data from relational database management systems.
- SQL can execute queries against the database.
- SQL is used to describe the data.
- SQL is used to define the data in the database and manipulate it when needed.
- SQL is used to create and drop the database and table.
- SQL is used to create a view, stored procedure, function in a database.
- SQL allows users to set permissions on tables, procedures, and views.

Advantages of SQL

There are the following advantages of SQL:

High speed

Using the SQL queries, the user can quickly and efficiently retrieve a large amount of records from a database.

No coding needed

In the standard SQL, it is very easy to manage the database system. It doesn't require a substantial amount of code to manage the database system.

Well defined standards

Long established are used by the SQL databases that are being used by ISO and ANSI.

Portability

SQL can be used in laptop, PCs, server and even some mobile phones.

Interactive language

SQL is a domain language used to communicate with the database. It is also used to receive answers to the complex questions in seconds.

Multiple data view

Using the SQL language, the users can make different views of the database structure

Disadvantages of SQL

Now let's talk about of the disadvantages of SQL.

1. Poor Interface

SQL has a poor interface as it makes look everything very complex even when it's not! Due to its difficult interfacing, users find it difficult to deal with the databases.

2. Cost Inefficient

SQL Server Standard costs around $1,418/year. The high cost makes it difficult for some programmers to use it.

3. Partial Control

SQL doesn't grant complete control over databases to its users. This is due to some hidden business rules.

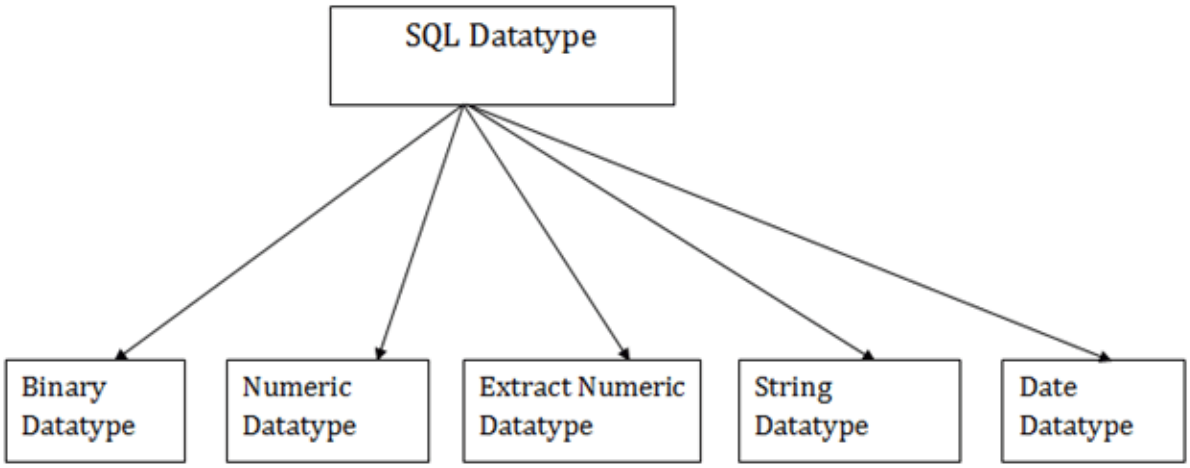
4. Security

Regardless of the SQL version, databases in SQL are constantly under threat as it holds huge amounts of sensitive data.

SQL Datatype

- SQL Datatype is used to define the values that a column can contain.
- Every column is required to have a name and data type in the database table.

Datatype of SQL:



1. Binary Datatypes

There are Three types of binary Datatypes which are given below:

Data Type	Description
binary	It has a maximum length of 8000 bytes. It contains fixed-length binary data.
varbinary	It has a maximum length of 8000 bytes. It contains variable-length binary data.
image	It has a maximum length of 2,147,483,647 bytes. It contains variable-length binary data.

2. Approximate Numeric Datatype :

The subtypes are given below:

Data type	From	To	Description
float	-1.79E + 308	1.79E + 308	It is used to specify a floating-point value e.g. 6.2, 2.9 etc.
real	-3.40e + 38	3.40E + 38	It specifies a single precision floating point number

3. Exact Numeric Datatype

The subtypes are given below:

Data type	Description
int	It is used to specify an integer value.
smallint	It is used to specify small integer value.
bit	It has the number of bits to store.
decimal	It specifies a numeric value that can have a decimal number.
numeric	It is used to specify a numeric value.

4. Character String Datatype

The subtypes are given below:

Data type	Description
char	It has a maximum length of 8000 characters. It contains Fixed-length non-unicode characters.
varchar	It has a maximum length of 8000 characters. It contains variable-length non-unicode characters.
text	It has a maximum length of 2,147,483,647 characters. It contains variable-length non-unicode characters.

5. Date and time Datatypes

The subtypes are given below:

Datatype	Description
date	It is used to store the year, month, and days value.
time	It is used to store the hour, minute, and second values.
timestamp	It stores the year, month, day, hour, minute, and the second value.

Note: Notes for examples of operator in SQL already provided in the classroom.

What is an Operator in SQL?

An operator is a reserved word or a character used primarily in an SQL statement's WHERE clause to perform operation(s), such as comparisons and arithmetic operations. These Operators are used to specify conditions in an SQL statement and to serve as conjunctions for multiple conditions in a statement.

- Arithmetic operators
- Comparison operators
- Logical operators
- Operators used to negate conditions

SQL Arithmetic Operators

Assume **'variable a'** holds 10 and **'variable b'** holds 20, then –

Show Examples

Operator	Description	Example

+ (Addition)	Adds values on either side of the operator.	a + b will give 30
- (Subtraction)	Subtracts right hand operand from left hand operand.	a - b will give -10
* (Multiplication)	Multiplies values on either side of the operator.	a * b will give 200
/ (Division)	Divides left hand operand by right hand operand.	b / a will give 2
% (Modulus)	Divides left hand operand by right hand operand and returns remainder.	b % a will give 0

SQL Comparison/Relational Operators

Assume **'variable a'** holds 10 and **'variable b'** holds 20, then **–**

Show Examples

Operator	Description	Example
=	Checks if the values of two operands are equal or not, if yes then condition becomes true.	(a = b) is not true.
!=	Checks if the values of two operands are equal or not, if values are not equal then condition becomes true.	(a != b) is true.
<>	Checks if the values of two operands are equal or not, if values are not equal then condition becomes true.	(a <> b) is true.
>	Checks if the value of left operand is greater than the value of right operand, if yes then condition becomes true.	(a > b) is not true.
<	Checks if the value of left operand is less than the value of right operand, if yes then condition becomes true.	(a < b) is true.
>=	Checks if the value of left operand is greater than or equal to the value of right operand, if yes then condition becomes true.	(a >= b) is not true.

<=	Checks if the value of left operand is less than or equal to the value of right operand, if yes then condition becomes true.	(a <= b) is true.
!<	Checks if the value of left operand is not less than the value of right operand, if yes then condition becomes true.	(a !< b) is false.
!>	Checks if the value of left operand is not greater than the value of right operand, if yes then condition becomes true.	(a !> b) is true.

SQL Logical Operators

Here is a list of all the logical operators available in SQL.

Show Examples

Sr.No.	Operator & Description
1	**ALL** The ALL operator is used to compare a value to all values in another value set.
2	**AND** The AND operator allows the existence of multiple conditions in an SQL statement's WHERE clause.
3	**ANY** The ANY operator is used to compare a value to any applicable value in the list as per the condition.
4	**BETWEEN** The BETWEEN operator is used to search for values that are within a set of values, given the minimum value and the maximum value.
5	**EXISTS** The EXISTS operator is used to search for the presence of a row in a specified table that meets a certain criterion.

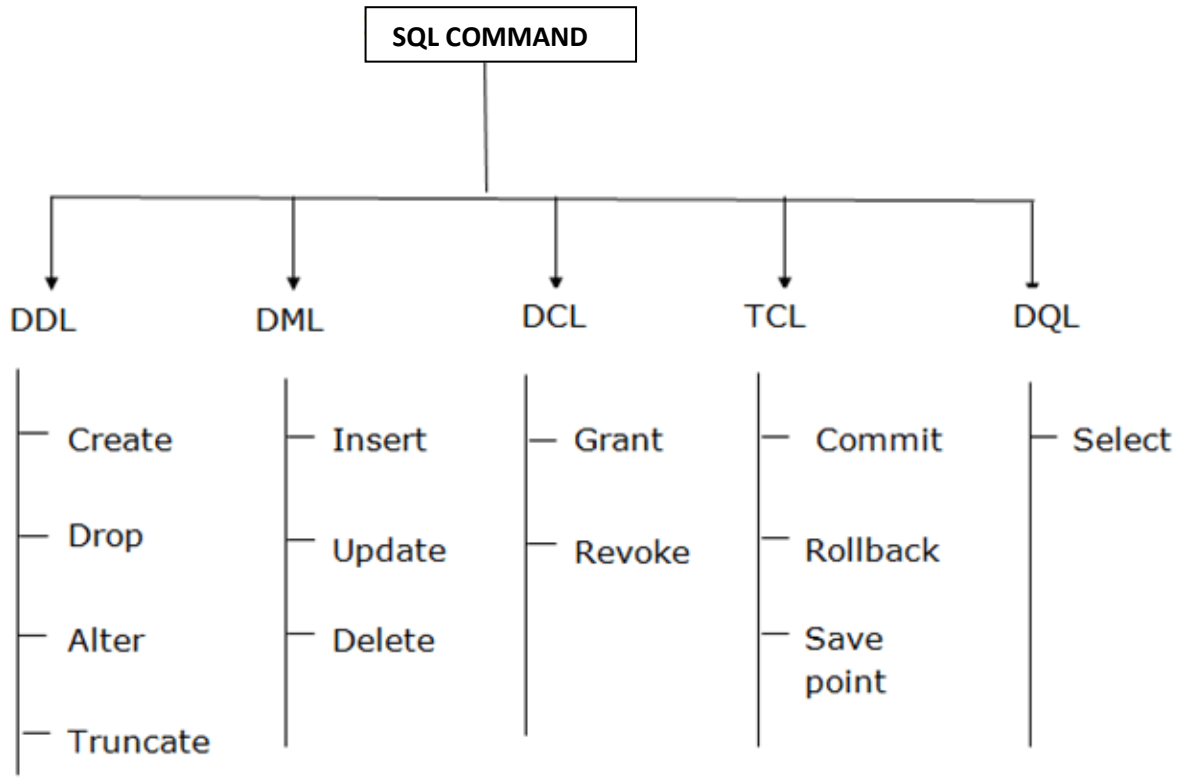
6	**IN** The IN operator is used to compare a value to a list of literal values that have been specified.
7	**LIKE** The LIKE operator is used to compare a value to similar values using wildcard operators.
8	**NOT** The NOT operator reverses the meaning of the logical operator with which it is used. Eg: NOT EXISTS, NOT BETWEEN, NOT IN, etc. **This is a negate operator.**
9	**OR** The OR operator is used to combine multiple conditions in an SQL statement's WHERE clause.
10	**IS NULL** The NULL operator is used to compare a value with a NULL value.
11	**UNIQUE** The UNIQUE operator searches every row of a specified table for uniqueness (no duplicates).

SQL Commands

- SQL commands are instructions. It is used to communicate with the database. It is also used to perform specific tasks, functions, and queries of data.
- SQL can perform various tasks like create a table, add data to tables, drop the table, modify the table, set permission for users.

Types of SQL Commands

There are five types of SQL commands: DDL, DML, DCL, TCL, and DQL.



1. Data Definition Language (DDL)

- DDL changes the structure of the table like creating a table, deleting a table, altering a table, etc.
- All the command of DDL are auto-committed that means it permanently save all the changes in the database.

Here are some commands that come under DDL:

- CREATE
- ALTER
- DROP
- TRUNCATE

a. CREATE It is used to create a new table in the database.

Syntax:

1. CREATE TABLE TABLE_NAME (COLUMN_NAME DATATYPES[,....]);

NAMING CONVENTIONS OF A TABLE:

We cannot give any name for the table. There are some rules for giving table name. A table name,

1. Must begin with a letter A_Z or a-z.
2. Can be from 1-30 characters long.
3. Cannot be SQL reserved word.
4. May contain underscore, letter, numericals & special character like A-Z, a-z,0-9,_,$ and # but not blank space & single quotes.
5. Is not case sensitive.
6. Must not duplicate the name of another table owned by the same user.
7. Table must be Unique.
8. Can have maximum 1000 character.

Example:

1. CREATE TABLE EMPLOYEE(Name VARCHAR2(20), Email VARCHAR2(100), DOB DATE);

b. DROP: It is used to delete both the structure and record stored in the table.

Syntax

1. DROP TABLE table_name;

Example

1. DROP TABLE EMPLOYEE;

c. ALTER: It is used to alter the structure of the database. This change could be either to modify the characteristics of an existing attribute or probably to add a new attribute.

Alter table command is mainly used under the following circumstances.

1. When a user wants to add new column
2. To change the Width or datatype itself of an existing column
3. To Enable or Disable the Constraints
4. Add the Constraint to an existing column.
5. Modify the Constraint or to drop the Constraint.

Syntax:

To add a new column in the table

1. ALTER TABLE table_name ADD column_name COLUMN-definition;
1. ALTER TABLE table_name MODIFY(column_definitions....);

EXAMPLE

1. ALTER TABLE STU_DETAILS ADD(ADDRESS VARCHAR2(20));
2. ALTER TABLE STU_DETAILS MODIFY (NAME VARCHAR2(20));

d. TRUNCATE: It is used to delete all the rows from the table and free the space containing the table.

Syntax:

1. TRUNCATE TABLE table_name;

Example:

1. TRUNCATE TABLE EMPLOYEE;

f. RENAME: To change the name of table,view,sequence, or synonym, You execute the rename statement.

Syntax:

Rename old_Name to New_Name;

Example:

Rename Emp to Employee;

2. Data Manipulation Language

- DML commands are used to modify the database. It is responsible for all form of changes in the database.
- The command of DML is not auto-committed that means it can't permanently save all the changes in the database. They can be rollback.

Here are some commands that come under DML:

- INSERT
- UPDATE
- DELETE

a. INSERT: The INSERT statement is a SQL query. It is used to insert data into the row of a table.

Syntax:

1. INSERT INTO TABLE_NAME
2. (col1, col2, col3,.... col N)
3. VALUES (value1, value2, value3, ... valueN);

Or

INSERT INTO TABLE_NAME
VALUES (value1, value2, value3, ... valueN);

For example:

INSERT INTO students (RollNo, FIrstName, LastName) VALUES ('60', 'Tom', Erichsen');

UPDATE:

This command is used to update or modify the value of a column in the table.

Syntax:

UPDATE table_name SET [column_name1= value1,...column_nameN = valueN] [WHERE CONDITION]

For example:

UPDATE students
SET FirstName = 'Jhon', LastName= 'Wick'
WHERE StudID = 3;

DELETE:

This command is used to remove one or more rows from a table.

Syntax:

DELETE FROM table_name [WHERE condition];

For example:

```
DELETE FROM students
WHERE FirstName = 'Jhon';
```

DCL:

DCL (Data Control Language) includes commands like GRANT and REVOKE, which are useful to give “rights & permissions.” Other permission controls parameters of the database system.

Examples of DCL commands:

Commands that come under DCL:

- Grant
- Revoke

Grant:

This command is use to give user access privileges to a database.

Syntax:

```
GRANT SELECT, UPDATE ON MY_TABLE TO SOME_USER, ANOTHER_USER;
```

For example:

```
GRANT SELECT ON Student TO BCA;
```

Revoke:

It is useful to back permissions from the user.

Syntax:

```
REVOKE privilege_nameON object_nameFROM {user_name |PUBLIC |role_name}
```

For example:

```
REVOKE SELECT, UPDATE ON student FROM BCA
```

TCL:

Transaction control language or TCL commands deal with the transaction within the database.

Commit

- All the commands that are executed consecutively, treated as a single unit of work and termed as a **transaction.**
- If you want to save all the commands which are executed in a transaction, then just after completing the transaction, you have to execute the ***commit*** command. This command will save all the commands which are executed on a table. All these changes made to the table will be saved to the disk permanently.
- Whenever the commit command is executed in SQL, all the updations which we have carried on the table will be uploaded to the server, and hence our work will be saved.

Syntax:

```
Commit;
```

For example:

```
DELETE FROM Students
WHERE RollNo =25;
COMMIT;
```

Rollback

- The ***rollback*** command is used to get back to the previous permanent status of the table, which is saved by the commit command.
- Suppose, we have started editing a table and later thought that the changes that we have recently carried out on a table are not required. Then, in that case, we can roll back our transaction, which simply means to get back to the previous permanent status of the table, which is saved by the commit command.

Syntax:

```
ROLLBACK;
```

Example:

```
DELETE FROM Students
WHERE RollNo =25;
```

SAVEPOINT

- Savepoint is a command in SQL that is used with the rollback command.
- It is a command in Transaction Control Language that is used to mark the transaction in a table.
- Consider you are making a very long table, and you want to roll back only to a certain position in a table then; this can be achieved using the savepoint.
- If you made a transaction in a table, you could mark the transaction as a certain name, and later on, if you want to roll back to that point, you can do it easily by using the transaction's name.
- Savepoint is helpful when we want to roll back only a small part of a table and not the whole table. In simple words, we can say savepoint is a bookmark in SQL.

Syntax:

```
SAVEPOINT SAVEPOINT_NAME;
```

```
Example: savepoint A;
```

What is DQL?

The SQL **SELECT** statement is used to fetch the data from a database table which returns this data in the form of a result table. These result tables are called result-sets.

The SQL **SELECT** statement is used to fetch the data from a database table which returns this data in the form of a result table. These result tables are called result-sets.

Syntax

The basic syntax of the SELECT statement is as follows –

```
SELECT column1, column2, columnN FROM table_name;
```

Here, column1, column2... are the fields of a table whose values you want to fetch. If you want to fetch all the fields available in the field, then you can use the following syntax.

```
SELECT * FROM table_name;
```

Example:

```
SQL> SELECT * FROM CUSTOMERS;
```

SQL - WHERE Clause:

The SQL **WHERE** clause is used to specify a condition while fetching the data from a single table or by joining with multiple tables. If the given condition is satisfied, then only it returns a specific value from the table. You should use the WHERE clause to filter the records and fetching only the necessary records.

The WHERE clause is not only used in the SELECT statement, but it is also used in the UPDATE, DELETE statement, etc.

Syntax

The basic syntax of the SELECT statement with the WHERE clause is as shown below.

```
SELECT column1, column2, columnN
FROM table_name
WHERE [condition]
```

Example:

```
SELECT ID, NAME, SALARY
FROM CUSTOMERS
WHERE SALARY > 2000;
```

SQL - ORDER BY Clause

The SQL **ORDER BY** clause is used to sort the data in ascending or descending order, based on one or more columns. Some databases sort the query results in an ascending order by default.

Syntax

The basic syntax of the ORDER BY clause is as follows –

```
SELECT column-list
FROM table_name
[WHERE condition]
[ORDER BY column1, column2, .. columnN] [ASC | DESC];
```

Example:

```
SELECT * FROM CUSTOMERS
  ORDER BY NAME DESC;
```

SQL - Group By

The SQL **GROUP BY** clause is used in collaboration with the SELECT statement to arrange identical data into groups. This GROUP BY clause follows the WHERE clause in a SELECT statement and precedes the ORDER BY clause.

Syntax

The basic syntax of a GROUP BY clause is shown in the following code block. The GROUP BY clause must follow the conditions in the WHERE clause and must precede the ORDER BY clause if one is used.

```
SELECT column1, column2
FROM table_name
WHERE [ conditions ]
GROUP BY column1, column2
ORDER BY column1, column2
```

Example:

```
SQL> SELECT NAME, SUM(SALARY) FROM CUSTOMERS
  GROUP BY NAME;
```

HAVING Clause in SQL

The HAVING clause places the condition in the groups defined by the GROUP BY clause in the SELECT statement.

This SQL clause is implemented after the 'GROUP BY' clause in the 'SELECT' statement.

This clause is used in SQL because we cannot use the WHERE clause with the SQL aggregate functions. Both WHERE and HAVING clauses are used for filtering the records in SQL queries.

Syntax of HAVING clause in SQL

SELECT column_Name1, column_Name2, ..., column_NameN aggregate_function_name(column_Name) **FROM** table_name **GROUP BY** column_Name1 **HAVING** condition;

Example:

SELECT SUM(Emp_Salary), Emp_City **FROM** Employee **GROUP BY** Emp_City **HAVING** SUM(Emp_Salary)>5000;

SQL Integrity Constraints

Integrity Constraints are used to apply business rules for the database tables.

The constraints available in SQL are **Foreign Key**, **Not Null**, **Unique**, **Check**.
Constraints can be defined in two ways
1) The constraints can be specified immediately after the column definition. This is called column-level definition.
2) The constraints can be specified after all the columns are defined. This is called table-level definition.

1) SQL Primary key:

This constraint defines a column or combination of columns which uniquely identifies each row in the table.

Syntax to define a Primary key at column level:

```
column name datatype [CONSTRAINT constraint_name] PRIMARY KEY
```

Syntax to define a Primary key at table level:

```
[CONSTRAINT constraint_name] PRIMARY KEY (column_name1,column_name2,..)
```

- **column_name1, column_name2** are the names of the columns which define the primary Key.
- The syntax within the bracket i.e. [CONSTRAINT constraint_name] is optional.

For Example: To create an employee table with Primary Key constraint, the query would be like.

Primary Key at column level:

```
CREATE TABLE employee
( id number(5) PRIMARY KEY,
name char(20),
dept char(10),
```

```
age number(2),
salary number(10),
location char(10)
);
```

or

```
CREATE TABLE employee
( id number(5) CONSTRAINT emp_id_pk PRIMARY KEY,
name char(20),
dept char(10),
age number(2),
salary number(10),
location char(10)
);
```

Primary Key at column level:

```
CREATE TABLE employee
( id number(5),
name char(20),
dept char(10),
age number(2),
salary number(10),
location char(10),
```

```
CONSTRAINT emp_id_pk PRIMARY KEY (id)
);
```

Primary Key at table level:

```
CREATE TABLE employee
( id number(5), NOT NULL,
name char(20),
dept char(10),
age number(2),
salary number(10),
location char(10),
ALTER TABLE employee ADD CONSTRAINT PK_EMPLOYEE_ID PRIMARY KEY
(id)
);
```

2) SQL Foreign key or Referential Integrity :

This constraint identifies any column referencing the PRIMARY KEY in another table. It establishes a relationship between two columns in the same table or between different tables. For a column to be defined as a Foreign Key, it should be a defined as a Primary Key in the table which it is referring. One or more columns can be defined as Foreign key.

Syntax to define a Foreign key at column level:

```
[CONSTRAINT constraint_name] REFERENCES
Referenced_Table_name(column_name)
```

Syntax to define a Foreign key at table level:

```
[CONSTRAINT constraint_name] FOREIGN KEY(column_name)
REFERENCES referenced_table_name(column_name);
```

For Example:

1) Lets use the "product" table and "order_items".

Foreign Key at column level:

```
CREATE TABLE product
( product_id number(5) CONSTRAINT pd_id_pk PRIMARY KEY,
product_name char(20),
supplier_name char(20),
unit_price number(10)
);

CREATE TABLE order_items
( order_id number(5) CONSTRAINT od_id_pk PRIMARY KEY,
product_id number(5) CONSTRAINT pd_id_fk REFERENCES,
product(product_id),
product_name char(20),
supplier_name char(20),
unit_price number(10)
);
```

Foreign Key at table level:

```
CREATE TABLE order_items
( order_id number(5) ,
product_id number(5),
product_name char(20),
supplier_name char(20),
unit_price number(10)
```

```
CONSTRAINT od_id_pk PRIMARY KEY(order_id),
CONSTRAINT pd_id_fk FOREIGN KEY(product_id) REFERENCES
product(product_id)
);
```

2) If the employee table has a 'mgr_id' i.e, manager id as a foreign key which references primary key 'id' within the same table, the query would be like,

```
CREATE TABLE employee
( id number(5) PRIMARY KEY,
name char(20),
dept char(10),
age number(2),
mgr_id number(5) REFERENCES employee(id),
salary number(10),
location char(10)
);
```

3) SQL Not Null Constraint :

This constraint ensures all rows in the table contain a definite value for the column which is specified as not null. Which means a null value is not allowed.

Syntax to define a Not Null constraint:

```
[CONSTRAINT constraint name] NOT NULL
```

For Example: To create a employee table with Null value, the query would be like

```
CREATE TABLE employee
( id number(5),
name char(20) CONSTRAINT nm_nn NOT NULL,
```

```
dept char(10),
age number(2),
salary number(10),
location char(10)
);
```

4) SQL Unique Key:

This constraint ensures that a column or a group of columns in each row have a distinct value. A column(s) can have a null value but the values cannot be duplicated.

Syntax to define a Unique key at column level:

```
[CONSTRAINT constraint_name] UNIQUE
```

Syntax to define a Unique key at table level:

```
[CONSTRAINT constraint_name] UNIQUE(column_name)
```

For Example: To create an employee table with Unique key, the query would be like,

Unique Key at column level:

```
CREATE TABLE employee
( id number(5) PRIMARY KEY,
name char(20),
dept char(10),
age number(2),
salary number(10),
location char(10) UNIQUE
);
```

or

```
CREATE TABLE employee
( id number(5) PRIMARY KEY,
name char(20),
dept char(10),
age number(2),
salary number(10),
location char(10) CONSTRAINT loc_un UNIQUE
);
```

Unique Key at table level:

```
CREATE TABLE employee
( id number(5) PRIMARY KEY,
name char(20),
dept char(10),
age number(2),
salary number(10),
location char(10),
CONSTRAINT loc_un UNIQUE(location)
);
```

5) SQL Check Constraint :

This constraint defines a business rule on a column. All the rows must satisfy this rule. The constraint can be applied for a single column or a group of columns.

Syntax to define a Check constraint:

```
[CONSTRAINT constraint_name] CHECK (condition)
```

For Example: In the employee table to select the gender of a person, the query would be like

Check Constraint at column level:

```
CREATE TABLE employee
( id number(5) PRIMARY KEY,
name char(20),
dept char(10),
age number(2),
gender char(1) CHECK (gender in ('M','F')),
salary number(10),
location char(10)
);
```

Check Constraint at table level:

```
CREATE TABLE employee
( id number(5) PRIMARY KEY,
name char(20),
dept char(10),
age number(2),
gender char(1),
salary number(10),
location char(10),
CONSTRAINT gender_ck CHECK (gender in ('M','F'))
);
```

SQL FUNCTION:

Arithmetic function

A mathematical function executes a mathematical operation usually based on input values that are provided as arguments, and return a numeric value as the result of the operation. Mathematical functions operates on numeric data such as decimal, integer, float, real, smallint, and tinyint.

By default, the precision of built-in operations on float data type data is six decimal places.

SQL arithmetic functions are:

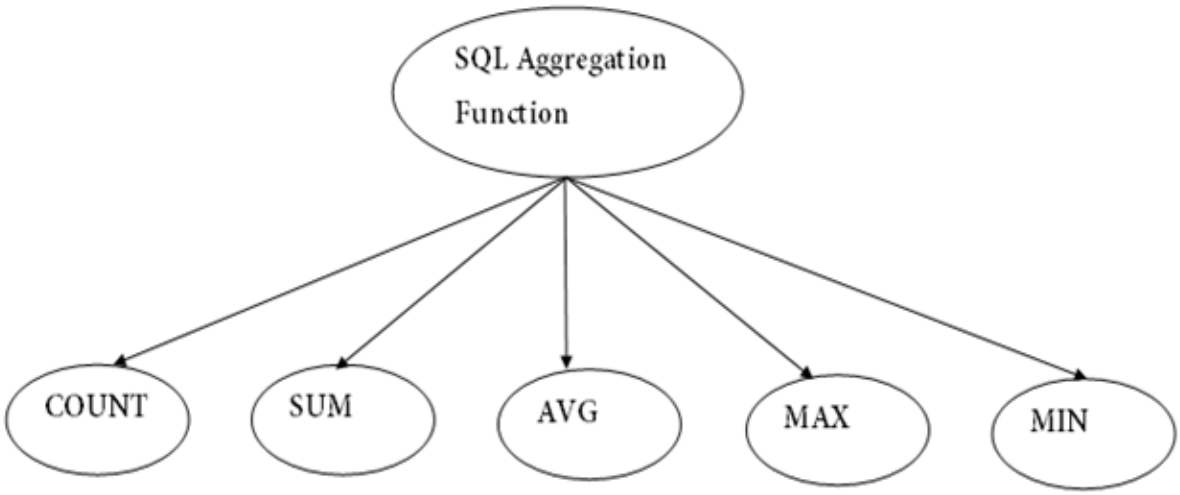
Functions	Description
ABS()	This SQL ABS() returns the absolute value of a number passed as an argument.
CEIL()	This SQL CEIL() will rounded up any positive or negative decimal value within the function upwards.
FLOOR()	The SQL FLOOR() rounded up any positive or negative decimal value down to the next least integer value.
EXP()	The SQL EXP() returns e raised to the n-th power(n is the numeric expression), where e is the base of natural algorithm and the value of e is approximately 2.71828183.
LN()	The SQL LN() function returns the natural logarithm of n, where n is greater than 0 and its base is a number equal to approximately 2.71828183.
MOD()	This SQL MOD() function returns the remainder from a division.

POWER()	This SQL POWER() function returns the value of a number raised to another, where both of the numbers are passed as arguments.
SQRT()	The SQL SQRT() returns the square root of given value in the argument.

SQL Aggregate Functions

- SQL aggregation function is used to perform the calculations on multiple rows of a single column of a table. It returns a single value.
- It is also used to summarize the data.

Types of SQL Aggregation Function



1. COUNT FUNCTION

- COUNT function is used to Count the number of rows in a database table. It can work on both numeric and non-numeric data types.
- COUNT function uses the COUNT(*) that returns the count of all the rows in a specified table. COUNT(*) considers duplicate and Null.

Syntax

1. COUNT(*)

2. or
3. COUNT([ALL|DISTINCT] expression)

Sample table:

PRODUCT_MAST

PRODUCT	COMPANY	QTY	RATE	COST
Item1	Com1	2	10	20
Item2	Com2	3	25	75
Item3	Com1	2	30	60
Item4	Com3	5	10	50
Item5	Com2	2	20	40
Item6	Cpm1	3	25	75
Item7	Com1	5	30	150
Item8	Com1	3	10	30
Item9	Com2	2	25	50
Item10	Com3	4	30	120

Example: COUNT()

1. SELECT COUNT(*) FROM PRODUCT_MAST;

Output:

```
10
```

2. SUM Function

Sum function is used to calculate the sum of all selected columns. It works on numeric fields only.

Syntax

1. SUM()
2. or
3. SUM([ALL|DISTINCT] expression)

Example: SUM()

1. SELECT SUM(COST)
2. FROM PRODUCT_MAST;

Output:

```
670
```

3. AVG function

The AVG function is used to calculate the average value of the numeric type. AVG function returns the average of all non-Null values.

Syntax

1. AVG()
2. or
3. AVG([ALL|DISTINCT] expression)

Example:

1. SELECT AVG(COST)
2. FROM PRODUCT_MAST;

Output:

```
67.00
```

4. MAX Function

MAX function is used to find the maximum value of a certain column. This function determines the largest value of all selected values of a column.

Syntax

1. MAX()
2. or
3. MAX([ALL|DISTINCT] expression)

Example:

1. SELECT MAX(RATE)
2. FROM PRODUCT_MAST;

```
30
```

5. MIN Function

MIN function is used to find the minimum value of a certain column. This function determines the smallest value of all selected values of a column.

Syntax

1. MIN()
2. or
3. MIN([ALL|DISTINCT] expression)

Example:

1. SELECT MIN(RATE)
2. FROM PRODUCT_MAST;

Output:

```
10
```

character functions:

The character functions accept only Character type values and return in character and number both.

Type of Character Functions

There are Eight types of Character Functions in SQL.

1. CONCAT
2. SUBSTR
3. LENGTH
4. INSTR
5. LPAD
6. RPAD
7. REPLACE
8. TRIM

So, let's understand the working of each function one by one.

The CONCAT function joint two or more strings and display as a single string. We can use a minimum of two strings with the CONCAT function. For example, we have two strings like "OCP" and "TECHNOLOGY" and we want to display both strings together.

```
SQL> SELECT  CONCAT('OCP','TECHNOLOGY') FROM DUAL;
```

The output from the above query is "OCPTECHNOLOGY".

The SUBSTR function Retrieve a part of the string as specified the length in function, as you can see in the below output. As we have a string like "OCPTECHNOLOGY" and we just want to display only the first three characters of the string.

```
SQL> SELECT SUBSTR('OCPTECHNOLOGY',1,3) FROM DUAL;
```

So, from the above query, we get the "OCP" as output.

The LENGTH function calculates the length of a string as a numeric value. Suppose we have a string like "OCPTECHNOLOGY" and we just want to count the letters of the sting, so you can use the length function.

```
SQL> SELECT LENGTH('OCPTECHNOLOGY') FROM DUAL;
```

The output is 13.

The INSTR function shows the position of a given character in number. For example, you want to find out the position of "T" later from the "OCPTECHNOLOGY" string.

```
SQL> SELECT INSTR('OCPTECHNOLOGY','T') FROM DUAL;
```

Output from above query is 4.

The LPAD (Left Pad) function will change the length of your column as you want for the session like you want your salary column with 10 digits. So LPAD solve your problem for the current session. The LPAD shows the character value just right justified.

```
SQL> SELECT LPAD(SAL,10,'*') FROM EMP;
```

The RPAD funcation (Right pad) is just opposite of LPAD function which shows the character value just left justified.

```
SQL> SELECT RPAD(SAL,10,'*') FROM EMP;
```

The REPLACE function just replace the value, it is work same like MS WORD, Notepad etc.

```
SQL> SELECT REPLACE('BLACK AND BLUE','BL','J') FROM DUAL;
```

In above query i just replace the 'BL' with 'J' and the output is "JACK AND JUE".
The TRIM function just trim a character or heading or both from a string.

```
SQL> SELECT TRIM('O' FROM 'OCPTECHNOLOGY') FROM DUAL;
```

From the above query, the TRIM function just trims the 'O' letter from 'OCPTECHNOLOGY' and you get 'CPTECHNOLOGY' as output.

Date Functions of SQL

In SQL, we have many data types available, which we can use as the date in our table. Some of them popularly being- 'YYYY-MM-DD' and 'DD-MM-YYYY'.

Formats of Date Time in SQL:
DATE – YYYY-MM-DD
DATETIME – YYYY-MM-DD HH:MI:SS
TIMESTAMP – YYYY-MM-DD HH:MI:SS
YEAR – YYYY or YY

Why do we need Date and Time Functions?

We have plenty of functions available for date and time in SQL. These are provided to make sure smooth access of the date and time module while making and accessing a SQL database.

Some of the most popular date and time functions are as follows:

Sr.No	Function	Description
1	NOW( )	Displays the current date and time.
2	CURDATE( )	Displays the current date.
3	CURTIME( )	Displays the current time.
4	DATE( )	Displays the date from the Date/DateTime expression.
5	EXTRACT( )	Displays selected part i.e. date/time.
6	DAY( )	Displays the day from the given date.
7	MONTH( )	Displays the month from the given date.
8	YEAR( )	Displays the year from the given date.
9	DATE_ADD( )	Displays date after adding the given interval.
10	DATE_SUB( )	Displays date after subtracting the given interval.
11	DATEDIFF( )	Displays the interval between two dates.
12	DATE_FORMAT( )	Displays the date/time data in various formats available.

Conversion Functions in SQL

There may be times when the SQL query was expecting some data type but it got another. In that case type conversion is done by SQL. This is implicit type conversion. However, sometimes the programmer explicitly converts one data type into another in a query. This is known as explicit type conversion. Both of these in detail are given as follows:

Implicit Data Type Conversion

In implicit data type conversion, the SQL programmer does not specify anything. Rather the system converts one type of data to another as per its requirements. For example - a numeric data type can be converted to character if required and vice versa.

Implicit DataType Conversion only occurs if the data types used in the query are valid so that the system can recognise them and convert them as required. This is not possible with invalid or wrongly provided data types.

Explicit Data Type Conversion

If the programmer wants,they can explicitly convert data types from one form to another. There are SQL functions provided for this express purpose. These SQL functions are –

TO_CHAR

This function is used to explicitly convert a number or date data type to char.

The syntax of this function is given as follows –

```
TO_CHAR(number,format,parameters)
```

This function changes number to char with the specific format as provided according to the syntax. The parameters can be used to specify decimal characters, group separators etc.

For example –

```
SELECT CHAR(sysdate, "Month DD,YYYY") FROM DUAL;
```

This returns the system date in the form of 31 July, 2018

TO_NUMBER

This function is used to explicitly convert a string to number. If the string to be converted does not contain numeric characters then TO_NUMBER shows an error.

The syntax of this function is given as follows –

```
TO_NUMBER(number,format,parameters)
```

This function changes the given string to number with the specific format as provided according to the syntax.

The parameters and format are optional so to convert a string to a number

For example,

```
TO_NUMBER ('353.78');
```

This returns the string 353.78 in numerical format.

TO_DATE

This function takes character values and returns the output in the date format.

The syntax of this function is given as follows –

```
TO_DATE(number, format, parameters)
```

This function changes the given string to number with the specific format as provided according to the syntax.

```
SELECT TO_DATE('2018/07/31','yyyy/mm/dd') FROM DUAL;
```